

Sleeping Machines

Computing with races, cancellations and near misses · status report, 25 September 2026

In one page

Sleeping Machines proposes that computation can happen **in time rather than memory**: candidate events race, the first to fire cancels the rest, and the cancelled ones keep a trace of how close they came, which a later teaching signal can use. Every experiment below had its predictions written down before evaluation, and the report includes what did not work.

What holds up

- **A cancelled node can be taught (E4).** Keeping a loser's distance to threshold lets a delayed teacher promote the right answer. At $K = 128$ classes: 0.79 accuracy where the reward-modulated rule is at chance, with 6% of the weight updates of uniform credit.
- **Races decide as fast as the evidence allows (E2).** An accumulator race beats a fixed-time decoder at every decision time and tracks the optimal MSPRT, using only additions and a threshold; 74% of input events are never processed.
- **Learning work tracks activity, not capacity (E5).** With capacity grown $64\times$, the race's work stays flat, and it needs about **$300\times$ fewer weight updates** than a sparse softmax on the same connectivity. In round 2 its accuracy also caught up (0.998 vs 0.998 at 16k classes).
- **Local, event-driven learning reaches about 96% on MNIST (E6, round 3, one seed).** Counterfactual credit 95.9%, fired-only 96.1%, against 89.5% for a frozen random hidden layer: learning in the hidden layer adds about 6 points.

What does not (yet)

- **The counterfactual part of hidden credit adds nothing measurable** over the fired-only ablation (95.9% vs 96.1%). The single-layer control under the round-3 settings is still queued.
- **Energy.** Only the single racing layer is cheaper than an equally accurate dense model (about $2.4\times$ at inference). The hidden-layer networks use about 108k synaptic events per image, about $4\times$ the multiply-accumulates of a 32-unit MLP that is as accurate.
- **Statistics.** All E6 numbers are single seeds.

What looks promising

- Learning cost that scales with *errors and activity* rather than with model size (E4, E5), which is exactly what a learner living in a stream needs.
- **E7**, designed and queued: the race network learning from one causal stream with no epochs or batches, scarce and late labels, labels it asks for itself, and state carried across frames.

The idea

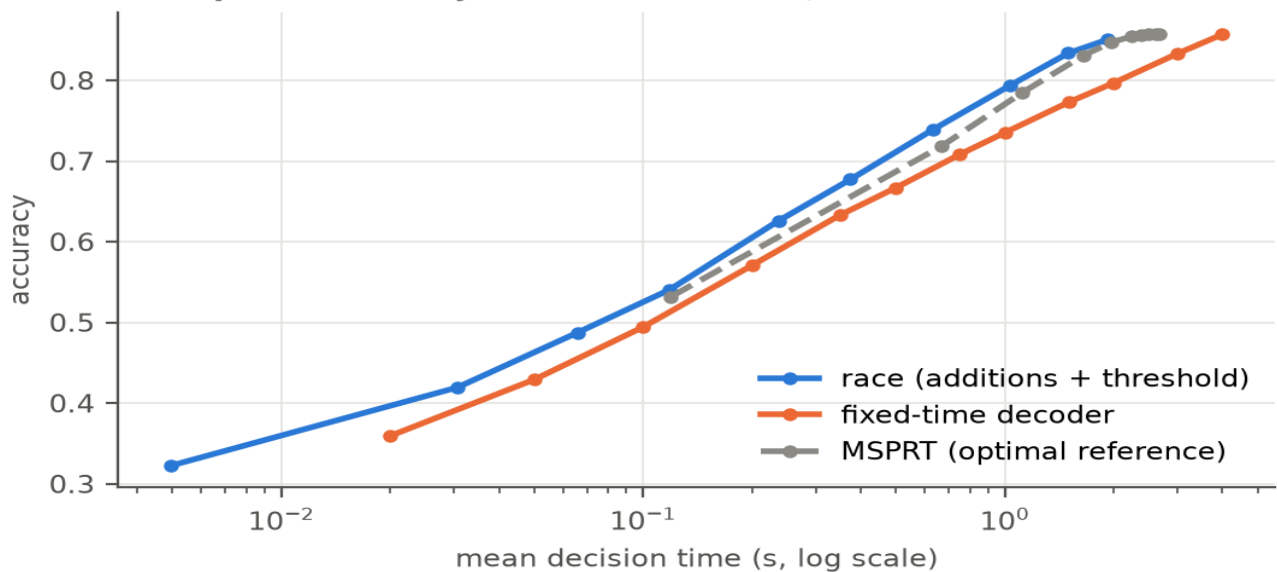
One race: B fires, A and C are cancelled but keep their distance to threshold



Nodes are non-leaky integrate-to-threshold units. The first to reach threshold fires and cancels the others; each cancelled node freezes Δ , its normalised distance to threshold. A teaching event arriving later can then credit near misses, not only the node that fired. The simulator has no global clock: it jumps from event to event and counts every operation, and all work and energy figures come from those counts.

E2 · decisions that take as long as they need

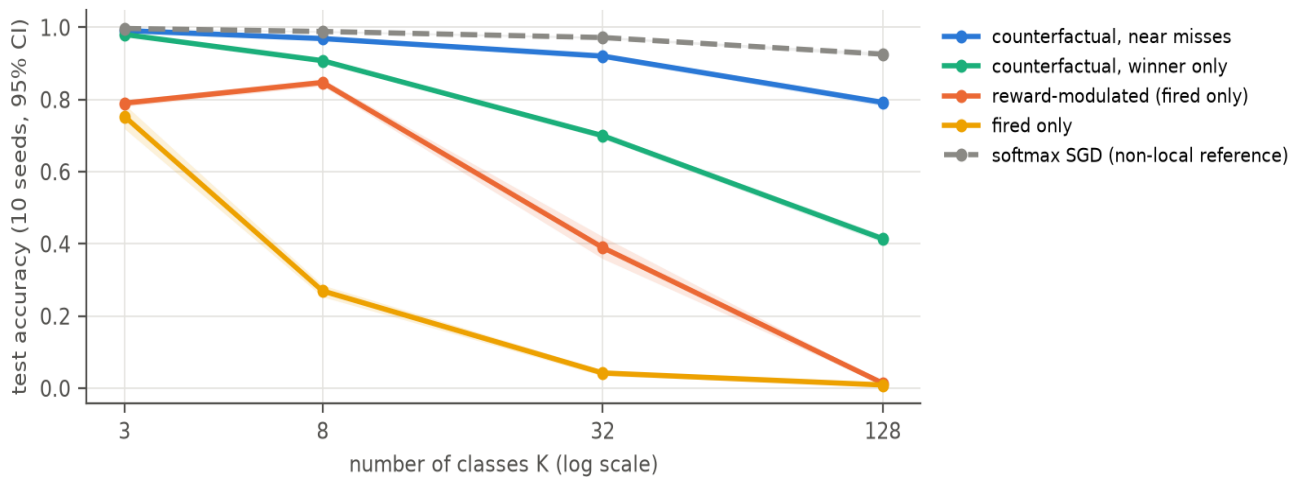
E2 · speed-accuracy frontier (4 classes, Poisson evidence)



The race dominates a fixed-time decoder at every matched decision time and follows the MSPRT, which knows the exact likelihoods. Easy trials end in 0.29 s, hard ones in 1.69 s, with no controller deciding when to stop.

E4 · counterfactual credit

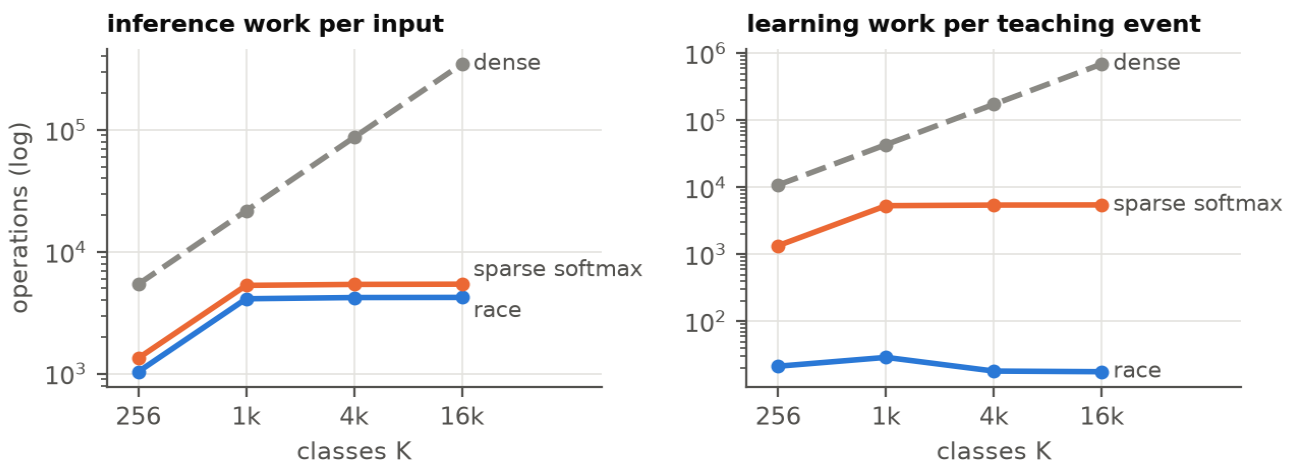
E4 · a cancelled node can be taught



Rules that credit only nodes that fired can only punish a wrong winner; as K grows the right answer almost never fires, and they collapse to chance. Counterfactual rules can promote the target from its frozen record. Crediting only competitors that came close is both the most accurate and the cheapest local rule. It still trails the global-gradient reference by 13 points at K = 128, and credit survives a delayed teacher only until another race overwrites the record.

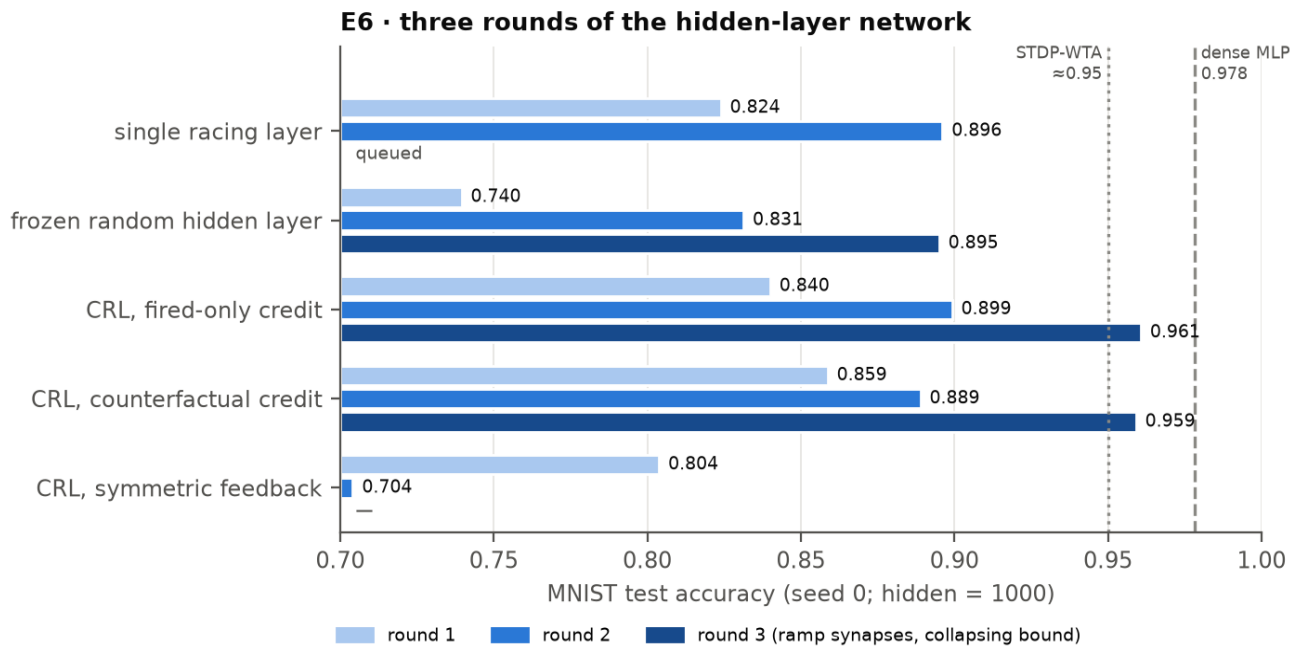
E5 · capacity that costs nothing while it sleeps

E5 round 2 · work as capacity grows 64× (3 seeds)



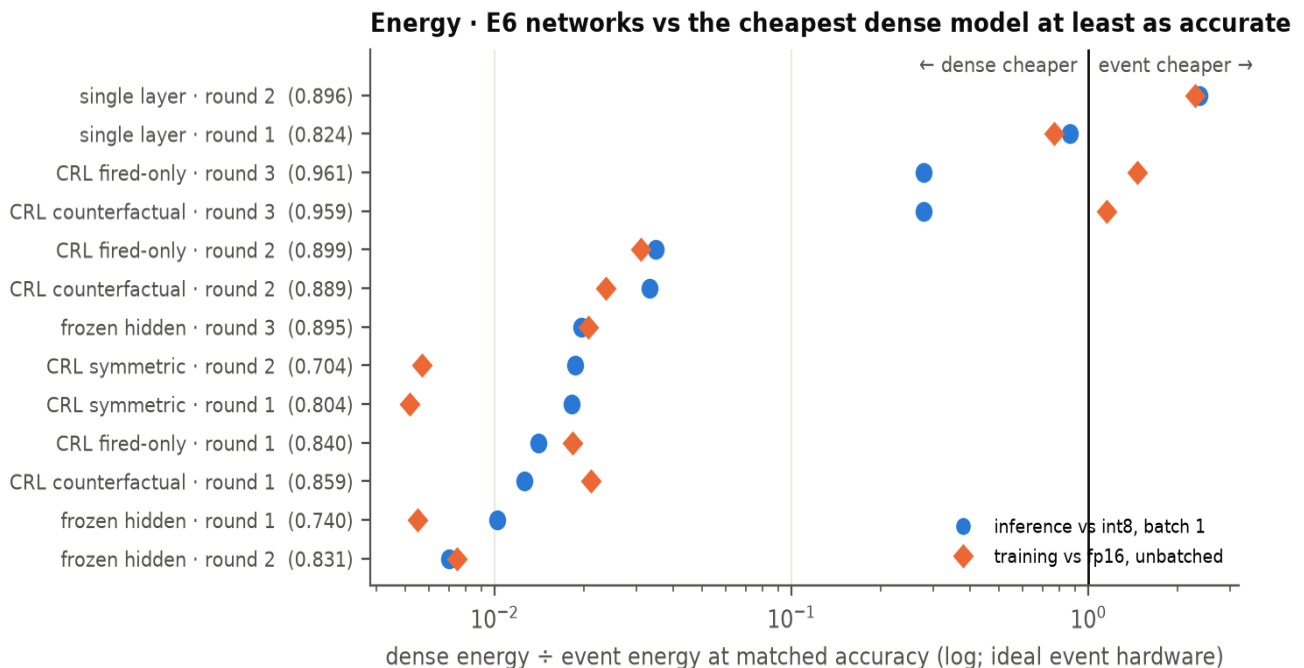
The fair baseline is a sparse softmax on the same connectivity, because most of what event computing saves comes from sparsity itself. Against it, inference work is similar (the race saves about 20% by stopping early), while learning work is about 300× lower: only near misses are updated. Dense models grow linearly with K.

E6 · hidden layers on MNIST



- Round 3 (current-based ramp synapses, a collapsing decision bound, 3 winners per group) lifted the hidden networks from about 0.89–0.90 to about 0.96.
- Hidden learning matters: the frozen random hidden layer stays at 0.895.
- Which hidden credit does not matter yet: fired-only $\approx$ counterfactual. Symmetric feedback was worst in rounds 1–2 and was not carried forward.
- A wider hidden layer (2000 nodes): running, result pending. The round-3 single-layer control is queued; until it is in, part of the round-3 gain may come from the new synapse model rather than from depth.

Energy, measured by counting



Operation counts priced with published per-operation energies (45 nm logic and SRAM; measured Loihi). These are order-of-magnitude estimates, not chip measurements. At inference, the single racing layer is the only network that beats an equally accurate dense model; the round-3 hidden networks are 1.2–1.5× cheaper to *train* than unbatched dense training. Hidden layers integrate about 100 input

events in each of 1000 nodes before inhibition stops them, and that swamps the savings. Against batched dense hardware every advantage disappears.

E7 · a learner that lives in a causal stream

Everything above trained the way clocked hardware likes: minibatches, IID epochs, a global learning-rate schedule, a label for every sample, a separate test phase. None of that is needed by an event-driven learner; it is inherited. E7 drops it. The guiding rule is not biological fidelity but this: **where we deviate from biology only because of synchronous-hardware habits, we do not carry the deviation over.**

frames (views of one image) → dark = label arrives orange bar = episode reset



slow state carried across frames: prior · tags · carried label · continuity · homeostasis · consolidation

fast state (one race per frame) restarts at every frame, so the exact solver still applies

each frame is predicted before it may be learned from (prequential); no epochs, no batches

habit	origin	in E7
minibatch updates	GPU throughput	one update per frame (Stage 0 checks this changes nothing)
epochs, reshuffling	batched SGD	one pass over a stream
global learning-rate decay	SGD convergence	per-synapse consolidation
homeostasis from batch means	vectorised statistic	per node, every frame, label or not
dense feedback matrix	cheap matmul	counted as events to eligible nodes only
train phase, then test	benchmark protocol	predict every frame before learning from it
a label for every sample	benchmark convention	10% labels, late labels, or labels the learner asks for
frames with a global $t = 0$	datasets are frames	kept as episode resets; slow state crosses them
latency-coded dense images	MNIST is a frame	kept as a benchmark, not a sensor model

Hypotheses (preregistered)

- **H0** one frame per update learns as well as minibatches of 32.
- **H1** a carried prior cuts the input events needed to decide on later views, without hurting first views.
- **H2** at 10% labels, carried labels and continuity help on real episodes and not on the **shuffled-time control** (same frames, order destroyed).
- **H3** labels the learner asks for (when its race is uncertain) beat random labels on the same budget.
- **H4** synaptic tags make labels that arrive 2 frames late usable.
- **H5** on class-blocked streams the race forgets less than an MLP trained by SGD; replay is reported alongside and may beat both.

Status. Code, tests and the pilot queue are in place; the pilots run after the E6 jobs finish. Pilots use the validation split to set thresholds, which are written into the preregistration before 5-seed confirmatory runs on the test set.

Lessons learned along the way

- **Compute discipline.** Parallel experiment runs hung the host twice (no swap; the second hard reboot corrupted the filesystem). The logs cannot say which job was responsible. Jobs now run strictly one at a time through a queue with a memory cap and a watchdog that stops everything if free memory falls below 3 GB.
- **Hidden batch assumptions.** E6 ran homeostasis inside the teaching step, which silently assumed every sample is taught. With scarce labels that would have switched homeostasis off. Batch-derived constants (homeostasis rate) also have to be rescaled when moving to one frame per update.
- **A case that batching hides.** A frame where no hidden node fires crashed the per-frame learner; in batches, some other sample always fired.
- **Undercounted training work.** The feedback fan-out and homeostasis updates were not counted. They are now, as events to eligible nodes only.
- **Controls before claims.** Round 3 looked like a win for counterfactual hidden credit until the fired-only ablation matched it. The single-layer and shuffled-time controls exist for the same reason.

Where this could go

- **Make the hidden layer cheap:** stop hidden integration as soon as the output race is decided, and use sparse (patch) connectivity, so hidden work stops dominating the energy budget.
- **Truly asynchronous inputs:** N-MNIST (event-camera recordings of MNIST) and Spiking Heidelberg Digits, which need several spikes per input channel.
- **Self-triggered resets:** treat a fixation or saccade the system itself initiates as the reset, so the frame boundary becomes a mechanism rather than a dataset artefact.
- **Seeds:** 3–5 seeds for every headline number before any claim.

Reproduce: `python report/make_pdf.py` rebuilds this document from `experiments/results` and `report/data.json`.
Preregistrations: `experiments/E*_PREREGISTRATION.md`.